ENG373 Dynamic Systems & Control — Two-Page Exam Sheet

Classical control, state space, estimation, optimisation and modern methods

● Classical/Modelling ● PID ● Frequency ● State Space ● Observer ● LQR ● Exam Logic

Exam Map & Answer Strategy ●

Format: 2 h, 100 marks, answer all; diagrams encouraged. Current coverage:

Q1 Modelling/stability: DC motor TF, poles, step response.
Q2 PID/robustness: advantages, margins, nonlinear limits.
Q3 Frequency domain: GM/PM definitions and robustness.
Q4 Controllability/observability: tests and limitations.
Q5 State feedback: pole placement, observer, practicality.
Q6 LQR: effect of R, optimality, advantage over PID.

For "explain": definition → physical cause → consequence. One clear sentence is better than an unexplained formula. Label axes, poles, zeros and feedback signs on sketches.

Control Foundations ●

$e(t) = r(t) - y(t)$ tracking error; controller acts to reduce it.

Open loop: output is not measured; simple/cheap, but cannot correct disturbances, uncertainty or calibration error.
Closed loop: measured output is fed back; improves tracking/disturbance rejection and reduces parameter sensitivity, but bad design can oscillate or destabilise.

$T(s) = \dfrac{G_cG}{1+G_cGH}$ negative-feedback closed-loop TF.

$S(s) = \dfrac{1}{1+L(s)}$ sensitivity; disturbance rejection.

$T(s) = \dfrac{L(s)}{1+L(s)}$ complementary sensitivity; reference tracking.

$S + T = 1$ fundamental robustness trade-off.

Robustness: acceptable stability/performance despite model error, parameter variation, disturbances and noise. Robust stability only guarantees stability; robust performance also preserves specifications.

Transfer Functions, Blocks & Laplace ●

$G(s) = \dfrac{Y(s)}{U(s)}$ LTI input-output model with zero initial conditions.

$G = C(sI-A)^{-1}B + D$ state-space to transfer function.

TF gives: poles, zeros, order, input-output dynamics. **TF hides:** internal states/physical realisation and nonzero initial energy.

$G_{series} = G_1G_2$ valid only with no loading interaction.

$G_{parallel} = G_1 + G_2$ outputs add.

$G_{fb} = \dfrac{G}{1 \pm GH}$ use sign at summing junction.

$1 \quad 1/s; \quad t \leftrightarrow 1/s^2; \quad \delta(t) \leftrightarrow 1$

$e^{-at} \quad 1/(s+a); \quad \sin\omega t \leftrightarrow \omega/(s^2+\omega^2)$

$\cos\omega t \quad s/(s^2+\omega^2); \quad \dot f \leftrightarrow sF - f(0)$

$y(\infty) = \lim_{s\to 0} sY(s)$ final-value theorem; only when applicable/stable.

Q1: DC Motor Modelling ●

$Li + Ri + K_b\omega = v$ electrical equation; back EMF opposes voltage.

$J\dot\omega + b\omega = K_ti - T_L$ mechanical equation; load torque is disturbance.

$$\frac{\Omega(s)}{V(s)} = \frac{K_t}{(Ls+R)(Js+b)+K_tK_b}$$

If armature inductance L is negligible, the fast electrical mode is removed:

$$\frac{\Omega}{V} \approx \frac{K}{\tau s+1},$$
$$K = \frac{K_t}{Rb+K_tK_b}, \qquad \tau = \frac{RJ}{Rb+K_tK_b}.$$

Logic: higher speed ⇒ larger back EMF ⇒ lower current/torque, so back EMF behaves like additional damping. Model assumptions must match the operating range.

Modelling Assumptions & Linearisation ●

A model is useful only over the conditions for which its assumptions are valid. State neglected dynamics, small-angle assumptions, constant parameters, linear friction, no saturation/dead-zone and operating range when relevant.

For nonlinear $\dot x = f(x,u)$, $y = h(x,u)$ around equilibrium (x_0, u_0):

$$\delta\dot x = A\delta x + B\delta u, \qquad \delta y = C\delta x + D\delta u,$$

$$A = \left.\frac{\partial f}{\partial x}\right|_0, \quad B = \left.\frac{\partial f}{\partial u}\right|_0, \quad C = \left.\frac{\partial h}{\partial x}\right|_0, \quad D = \left.\frac{\partial h}{\partial u}\right|_0.$$

Limitation: linearisation is local; large deviations or nonlinear constraints can invalidate the predicted stability/performance.

Test Inputs, System Type & Error Constants ●

Input	$R(s)$	what it tests
step	$1/s$	transient + final tracking
ramp	$1/s^2$	moving-reference tracking
impulse	1	natural modes
sinusoid	$\omega/(s^2+\omega^2)$	frequency response

For unity feedback with loop $L(s) = G_c(s)G(s)$:

$$K_p = \lim_{s\to 0} L(s), \qquad K_v = \lim_{s\to 0} sL(s), \qquad K_a = \lim_{s\to 0} s^2L(s).$$

$$e_{ss}^{step} = \frac{1}{1+K_p}, \qquad e_{ss}^{ramp} = \frac{1}{K_v}, \qquad e_{ss}^{parabolic} = \frac{1}{K_a}.$$

System type = number of pure integrators in $L(s)$. More integrators improve low-order tracking but add phase lag and reduce robustness.

Time Response & Pole Logic ●

$c(t) = c_{tr}(t) + c_{ss}(t)$ transient decays; steady-state remains.

$G_1(s) = \dfrac{1}{Ts+1}$ one stable pole at $-1/T$ for $T > 0$.

$c(t) = 1 - e^{-t/T}$ unit-step response: 63.2% at T, $t_s(2\%) \approx 4T$.

$G_2(s) = \dfrac{\omega_n^2}{s^2+2\zeta\omega_ns+\omega_n^2}$ standard second order.

$\zeta = 0$ sustained oscillation; imaginary-axis poles.
$0 < \zeta < 1$ decaying oscillation; LHP complex pair.
$\zeta = 1$ fastest non-oscillatory response.
$\zeta > 1$ slow monotonic response.

$M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}}100\%$ overshoot depends mainly on ζ.

$t_s \approx \dfrac{4}{\zeta\omega_n}$ 2% criterion; real part controls decay.

Poles: LHP stable; RHP unstable; farther left faster. Poles nearest the imaginary axis dominate. RHP zero may cause inverse initial response.

Q1: Stability & Pole Interpretation ●

Absolute stability: asks whether the system is stable or unstable.
Relative stability: asks how fast/well transients decay; indicated by damping, settling time, GM and PM.
BIBO stability: every bounded input produces a bounded output; a rational continuous-time TF is BIBO stable only when all poles are strictly in the LHP.

$\Re(s) < 0$ decaying mode ⇒ stable.
$\Re(s) = 0$ sustained mode; marginal only if imaginary-axis poles are simple.
$\Re(s) > 0$ growing mode ⇒ unstable.
$-\sigma \pm j\omega$ σ controls decay envelope; ω controls oscillation frequency.

$T(s) = \dfrac{G(s)}{1+G(s)H(s)}$ negative-feedback closed-loop transfer function.

$1 + G(s)H(s) = 0$ characteristic equation; its roots are the closed-loop poles.

Key exam sentence: Stability is determined by closed-loop poles, not by open-loop poles alone.

$V > 0$, $\dot V < 0$ Lyapunov direct method: asymptotically stable.

Routh–Hurwitz: Construction & Decision ●

For

$$a_ns^n + a_{n-1}s^{n-1} + \cdots + a_1s + a_0 = 0,$$

place alternating coefficients in the first two rows:

$$\begin{array}{c|cccc} s^n & a_n & a_{n-2} & a_{n-4} & \cdots \\ s^{n-1} & a_{n-1} & a_{n-3} & a_{n-5} & \cdots \\ s^{n-2} & b_1 & b_2 & b_3 & \cdots \end{array}$$

$$b_1 = \frac{a_{n-1}a_{n-2} - a_na_{n-3}}{a_{n-1}},$$
$$b_2 = \frac{a_{n-1}a_{n-4} - a_na_{n-5}}{a_{n-1}}.$$

Continue using the same two-row determinant pattern.
Main rule: number of sign changes in the first column = number of RHP roots. With positive leading coefficient, asymptotic stability requires every first-column term to be positive.
Fast pre-check: all polynomial coefficients must be present and have the same sign; this is necessary but not sufficient.
For

$$s^3 + a_1s^2 + a_2s + a_3,$$

stability requires

$$a_1 > 0, \qquad a_2 > 0, \qquad a_3 > 0, \qquad a_1a_2 > a_3.$$

Routh Special Cases & Gain Range ●

Zero first-column entry, row not zero: replace the zero by $\varepsilon > 0$, complete the array, take $\varepsilon \to 0^+$, then count sign changes. Never divide by the original zero.
Entire row of zeros: roots occur symmetrically about the origin, such as $\pm\sigma$, $\pm j\omega$, or complex quadruplets. Form the auxiliary polynomial $A(s)$ from the row above, replace the zero row by the coefficients of dA/ds, and continue.
Roots of $A(s) = 0$ identify the symmetric roots or imaginary-axis crossing.
Gain-range workflow:
1. Form $1 + KG(s)H(s) = 0$.
2. Build the Routh array symbolically.
3. Require every first-column entry to be positive.
4. Solve the resulting inequalities for K.

Equality gives a marginal-stability boundary.
Crossing frequency: at the limiting gain, use the auxiliary equation from the row above the zero row and solve for $s = \pm j\omega$.
Relative stability test: to require poles to lie left of $\Re(s) = -\alpha$, substitute

$$s = z - \alpha$$

and apply Routh to the polynomial in z.

Root Locus: Recognition & Design ●

$1 + KG(s)H(s) = 0$ closed-loop characteristic equation.

$\angle GH = (2k+1)180^\circ$ angle condition gives locus geometry.

$K = 1/|GH|$ magnitude condition gives gain at a point.

Rules: branches = number of OL poles; start at poles ($K = 0$), end at zeros/infinity; real-axis locus lies left of an odd number of real poles+zeros; symmetric about real axis.

$\sigma_a = \dfrac{\sum p - \sum z}{n-m}$ asymptote centroid.

$\phi_a = \dfrac{(2k+1)180^\circ}{n-m}$ asymptote angles.

$\dfrac{dK}{ds} = 0$ candidate breakaway/break-in; retain only points on locus.

$j\omega$ crossing comes from Routh.
Lead: zero closer to origin than pole; adds phase, pulls locus left, improves transient response/PM but can amplify HF noise.
Lag: pole closer to origin; raises low-frequency gain, reduces e_{ss}, adds phase lag and may slow response. Use lag-lead if both are needed.

Q2: PID Essentials ●

$$u = K_pe + K_i\int e\,dt + K_d\dot e, \qquad G_c = K_p + \frac{K_i}{s} + K_ds$$

	t_r	M_p	t_s	e_{ss}
K_p ↑	↓	↑	var.	↓
K_i ↑	↓	↓	↑	step → 0
K_d ↑	small	↓	↓	none

P: current error, faster but may destabilise. **I:** accumulated error, increases type and removes step error but adds phase lag. **D:** anticipates trend, adds damping/phase lead but amplifies noise.

PID Nonlinear/Practical Limits ●

Integrator windup: during saturation, commanded u cannot be delivered but the integral keeps growing, causing delayed recovery and overshoot. Use clamping, conditional integration or back-calculation.
Derivative kick: a step in setpoint makes $\dot e$ very large. Apply derivative to measured output rather than the setpoint error.
Noise: ideal derivative has high-frequency gain; use $K_ds/(1+s/N)$ or equivalent low-pass filter.
Saturation: controller is linear but actuator limits are nonlinear; performance and stability predictions may fail.
2DOF PID: P, D setpoint weights separate tracking from disturbance rejection; integral still acts on $r - y$.
Sampling: sample much faster than closed-loop bandwidth; too slow adds delay, too fast may reveal/amplify noise.

Ziegler–Nichols Tables ●

Method 1 — open-loop reaction curve: approximate $G(s) \approx Ke^{-Ls}/(Ts+1)$.

	K_p	T_i	T_d
P	$T/(KL)$	–	–
PI	$0.9T/(KL)$	$L/0.3$	–
PID	$1.2T/(KL)$	$2L$	$0.5L$

Method 2 — closed loop: P-only gain at sustained oscillation = K_u; period = P_u.

	K_p	T_i	T_d
P	$0.5K_u$	–	–
PI	$0.45K_u$	$P_u/1.2$	–
PID	$0.6K_u$	$P_u/2$	$P_u/8$

$K_i = K_p/T_i$, $K_d = K_pT_d$. ZN is a heuristic starting point (quarter-amplitude decay), commonly aggressive and requiring refinement.

Q3: Frequency Response & Bode Rules ●

For stable $G(s)$ and

$$u(t) = A\sin\omega t,$$

the steady-state output is

$$y_{ss}(t) = A|G(j\omega)|\sin(\omega t + \angle G(j\omega)).$$

$M_{dB} = 20\log_{10}|G(j\omega)|$ magnitude ratio in decibels.
$|G_1G_2|_{dB} = |G_1|_{dB} + |G_2|_{dB}$ products become sums on a Bode plot.

Factor	magnitude slope	phase
$K > 0$	constant $20\log_{10}K$	0°
$1/s$	-20 dB/dec	-90°
s	$+20$ dB/dec	$+90^\circ$
$1/(1+s/\omega_p)$	add -20 after ω_p	$0 \to -90^\circ$
$1+s/\omega_n$	add $+20$ after ω_z	$0 \to +90^\circ$
2nd-order pole	add -40 after ω_n	$0 \to -180^\circ$

At a first-order corner, the exact magnitude differs from the asymptote by 3 dB and the phase contribution is $\pm 45^\circ$.
Bandwidth: ω_B = frequency where the magnitude has dropped by 3 dB from the low-frequency value. Larger ω_B ⇒ faster closed-loop response.
Resonance: For a standard second-order system, a resonance peak occurs near $\omega \approx \omega_n$ when $\zeta < 0.707$. A second-order resonance peak occurs when

$$\zeta < \frac{1}{\sqrt{2}}.$$

Sketch workflow: factor G; list corner frequencies; set initial level and slope; change slope at each corner; add all phase contributions.

GM, PM, Nyquist & Robustness ●

Let

$$L(s) = G_c(s)G(s)H(s).$$

$|L(j\omega_{gc})| = 1$ (0 dB) gain crossover frequency.
$PM = 180^\circ + \angle L(j\omega_{gc})$ additional phase lag before marginal stability.
$\angle L(j\omega_{pc}) = -180^\circ$ phase crossover frequency.
$GM = \dfrac{1}{|L(j\omega_{pc})|}$ gain multiplication allowed before marginal stability.
$GM_{dB} = -20\log_{10}|L(j\omega_{pc})|$ distance from the magnitude curve to 0 dB at ω_{pc}.

Advantage: Encodes both magnitude AND phase in a single curve. **Nyquist low-frequency start:** Type 0 → finite DC gain; Type 1 → $-j\infty$; Type 2 → $-\infty$ on the real axis.
Lecture targets: typically

$$PM \approx 45^\circ\text{–}60^\circ, \qquad GM \geq 6 \text{ dB}.$$

Positive and larger margins usually indicate better tolerance to gain error, neglected dynamics and delay.
Delay tolerance:

$$t_d \approx \frac{\text{PM}\pi}{180\,\omega_{gc}}$$

Extra delay of approximately t_d reduces the phase margin to zero.
Nyquist convention used in the slides:

$$Z = N + P,$$

where:
P number of open-loop RHP poles.
N net clockwise encirclements of $-1 + j0$.
Z number of closed-loop RHP poles.

Closed-loop stability requires

$$Z = 0 \quad \Rightarrow \quad N = -P.$$

For an open-loop stable system, $P = 0$, so the Nyquist plot must not encircle -1.
Why is -1 critical? $L(j\omega) = -1$ makes

$$1 + L(j\omega) = 0,$$

placing the closed loop at marginal stability. A Nyquist curve close to -1 indicates weak robustness.
Delay and bandwidth: pure delay e^{-sT} leaves magnitude unchanged but adds phase

$$-\omega T.$$

Approximate delay margin:

$$T_{\max} \approx \frac{PM\ (\text{rad})}{\omega_{gc}}.$$

Higher bandwidth gives faster response, but increases sensitivity to noise, delay and unmodelled high-frequency dynamics. A -40 dB/dec slope near crossover is a warning sign.

State Space & Modern Control Map ● ● ●

Navigate: state models/tests → feedback/observers → LQR/Kalman/LQG → MPC, adaptive, learning, planning and safety. Use the coloured dots to connect definitions, workflows and comparison answers.

State-Space Meaning ●

$$\dot x = Ax + Bu, \qquad y = Cx + Du$$

$x \in \mathbb{R}^n$ minimum internal memory/state vector.
A $(n \times n)$ natural dynamics; eigenvalues are system poles.
B $(n \times m)$ how inputs affect state derivatives.
C $(p \times n)$ how states produce measured outputs.
D $(p \times m)$ direct input-output feedthrough.
Why state space? internal dynamics, MIMO, initial conditions, pole placement, observers and LQR. It is not unique: $z = Tx$ gives $A' = TAT^{-1}$, $B' = TB$, $C' = CT^{-1}$.

Eigenvalue & Lyapunov Stability ●

For $\dot x = Ax$, the system poles are the eigenvalues of A:

$$\det(sI - A) = 0, \qquad \lambda_i = \text{eig}(A).$$

Linear eigenvalue test:

$$\Re(\lambda_i) < 0\ \forall i \quad \Rightarrow \quad \text{asymptotically stable.}$$

Any eigenvalue with $\Re(\lambda_i) > 0$ makes the system unstable.
Lyapunov direct method: choose an energy-like scalar function $V(x)$ satisfying

$$V(0) = 0, \qquad V(x) > 0\ (x \neq 0), \qquad \dot V(x) < 0.$$

These conditions prove asymptotic stability without explicitly solving the differential equations.
For a linear system, choose $Q = Q^T > 0$ and solve

$$A^TP + PA = -Q.$$

A unique $P = P^T > 0$ exists for every $Q > 0$ if and only if A is Hurwitz.
Which method?

Routh	symbolic gain ranges and number of RHP roots.
Eigenvalues	exact pole locations and numerical MIMO analysis.
Lyapunov	energy-based proof; extends to nonlinear systems.

TF → State Space & Matrix Exponential ●

For

$$G(s) = \frac{b_0s^{n-1} + \cdots + b_{n-1}}{s^n + a_1s^{n-1} + \cdots + a_n},$$

controllable companion form:

$$A = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & & \ddots & \vdots \\ -a_n & -a_{n-1} & \cdots & -a_2 & -a_1 \end{bmatrix}.$$

$$B = [0 \quad 0 \quad \cdots \quad 1]^T$$

$$C = [b_{n-1} \quad b_{n-2} \quad \cdots \quad b_0], \qquad D = 0.$$

$$x(t) = e^{At}x(0) + \int_0^t e^{A(t-\tau)}Bu(\tau)\,d\tau.$$

If $A = V\Lambda V^{-1}$, then $e^{At} = Ve^{\Lambda t}V^{-1}$. Also $e^{At} = \mathcal{L}^{-1}\{(sI-A)^{-1}\}$.

Q4: Controllability ●

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix}$$

$$\text{rank}(\mathcal{C}) = n \iff \text{fully controllable.}$$

Meaning: some input exists that can move the state between arbitrary points in finite time. Columns span the reachable subspace.
Exam workflow: compute $AB, A^2B, \ldots$; assemble $\mathcal{C}$; row-reduce or use determinant for square SISO matrix; compare rank with n.
Design limitation: if rank $< n$, arbitrary pole placement is impossible; uncontrollable modes cannot be moved by K. They must at least be stable for stabilisability.
$\text{rank}[\lambda I - A \;\; B] = n$ PBH test for every $\lambda \in \text{eig}(A)$.

Q4: Observability ●

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

$$\text{rank}(\mathcal{O}) = n \iff \text{fully observable.}$$

Meaning: the initial/internal state can be reconstructed uniquely from known u and measured y over finite time.
Exam workflow: compute $CA, CA^2, \ldots$; stack them; row-reduce/determinant; compare rank with n.
Design limitation: if rank $< n$, hidden modes cannot be reconstructed by a full observer. They must at least be stable for detectability.
Duality: (A, C) observable ⇔ (A^T, C^T) controllable.
$\text{rank}\begin{bmatrix} \lambda I - A \\ C \end{bmatrix} = n$ PBH observability test.

Controllable vs Observable: Logic ●

	Controllable?	Observable?
Yes/Yes	place poles	estimate states; observer feedback feasible.
Yes/No	poles placeable if true states measured	output cannot reveal all states; observer impossible for hidden modes.
No/Yes	states can be estimated	some modes cannot be changed by input.
No/No	hidden and unreachable modes	only acceptable if those modes are stable.

Can you place poles if controllable but not observable? Yes with full-state measurement; no observer can reconstruct all states from the available output.
Can you estimate an uncontrollable system? Possibly; observability and controllability are separate properties.

Q5: State Feedback Pole Placement ●

$u = -Kx + Nr$ state feedback plus reference prefilter.
$\dot x = (A - BK)x + BNr$ closed-loop dynamics.
Procedure: (1) verify controllability; (2) choose desired poles; (3) expand desired polynomial $\alpha_c(s)$; (4) solve for K; (5) verify $\text{eig}(A - BK)$.
Ackermann (SISO):

$$K = [0 \quad \cdots \quad 0 \quad 1]\,\mathcal{C}^{-1}\phi(A),$$

$$\phi(A) = A^n + \alpha_1A^{n-1} + \cdots + \alpha_nI.$$

Practicality: poles too far left imply large gains, high actuator effort/saturation, noise sensitivity and model sensitivity. Pole placement has no built-in optimality or robustness objective.

Reference Tracking / Integral Augmentation

Plain $u = -Kx$ regulates states toward zero; it does not automatically make y track a nonzero reference or reject constant disturbances.
For a step reference, choose prefilter

$$N = \left[C(-A + BK)^{-1}B\right]^{-1}$$

when applicable, so nominal DC gain is one. A prefilter does not reject unknown constant disturbances/-model error.
For zero steady-state error robustly, augment an integral state:

$$\dot{x}_i = r - y, \qquad x_a = \begin{bmatrix} x \\ x_i \end{bmatrix},$$

then check controllability of the augmented system and design gains for all augmented states.

Why an Observer?

State feedback needs all components of x, but sensors may be unavailable, expensive, noisy or unreliable. An observer uses the model plus measured u, y to estimate unmeasured states.

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x})$$

The innovation $y - C\hat{x}$ corrects the model prediction. A full-order observer estimates all n states; a minimum-order observer estimates only unmeasured states (order $n - \operatorname{rank} C$ under suitable coordinates).

Observer Design & Separation

Let $e = x - \hat{x}$:

$$\dot{e} = (A - LC)e.$$

Choose L so $A - LC$ is stable and suitably fast; observability is required for arbitrary observer pole placement. By duality,

$$L = \operatorname{place}(A^T, C^T, p_{obs})^T.$$

Rule of thumb: observer poles about 2–5 times faster than controller poles. Too slow $\to$ delayed/poor estimates. Too fast $\to$ large L, noise amplification, actuator activity and model-error sensitivity.
Separation principle:

$$\operatorname{eig}(A_{combined}) = \operatorname{eig}(A - BK) \cup \operatorname{eig}(A - LC).$$

Thus K and L may be designed independently, then combined with $u = -K\hat{x} + Nr$.

Q6: LQR Core Workflow

$$J = \int_0^\infty \left(x^T Qx + u^T Ru\right) dt, \qquad u^* = -Kx$$

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

$$K = R^{-1}B^T P.$$

Requirements for standard infinite-horizon result: $Q \succeq 0$, $R \succ 0$, (A, B) stabilisable and penalised unstable modes detectable.
Matrix procedure: choose Q, R; assume symmetric P for hand calculation; substitute into ARE; equate matrix entries; select stabilising $P \succeq 0$; calculate K; verify $\operatorname{eig}(A - BK)$.

LQR Weight Logic / Bryson's Rule

$Q_{ii} \uparrow$ state x_i is expensive: tighter/faster regulation, usually higher control effort.
$R_{jj} \uparrow$ input u_j is expensive: smaller/conservative actuation, usually slower response.
Only the relative scale of Q and R matters strongly.
For systematic initial weights:

$$Q_{ii} = \frac{1}{(x_{i,\max})^2}, \qquad R_{jj} = \frac{1}{(u_{j,\max})^2}.$$

Then tune iteratively. Increasing R is the expected short-answer response to "what reduces control effort?"

LQR: Advantages, Robustness, Limits

Why optimal? K minimises the stated quadratic cost among admissible controls. Closed-loop poles emerge from the performance/effort trade-off rather than being assigned directly.
Advantages over PID: natural MIMO/state coupling; systematic matrix design; explicit control-effort trade-off; full-state SISO LQR guarantees at plant input: gain margin at least $[0.5, \infty)$ (-6 dB to $+\infty$) and phase margin at least $60°$.
PID advantages: simple, inexpensive, familiar, often needs less model/state information, easy for SISO loops.
LQR limitations: linear model, full state required, Q/R tuning still heuristic, no integral action by default, constraints/saturation not handled explicitly. Adding an observer gives LQG later, but the full-state LQR margin guarantees may be lost.

Pole Placement vs LQR

	Pole placement	LQR
Design input	desired poles	Q, R weights
Result	exact poles	poles emerge optimally
Cost	none explicit	minimises J
MIMO	possible but choices awkward	natural matrix formulation
Robustness	no inherent guarantee	full-state input-margin guarantees
Main risk	aggressive arbitrary poles	model/state dependence; weight tuning

Why not always place poles very far left? The response may be fast mathematically but require unrealistic gains, saturate actuators and amplify noise/unmodelled dynamics.

2×2 Matrix Mini-Example

Given

$$A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Controllability: $AB = \begin{bmatrix} 1 \\ -3 \end{bmatrix}$, $\mathcal{C} = \begin{bmatrix} 0 & 1 \\ 1 & -3 \end{bmatrix}$, $\det \mathcal{C} = -1 \neq 0$.
Observability: $CA = \begin{bmatrix} 0 & 1 \end{bmatrix}$, $\mathcal{O} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, rank $= 2$.
Pole placement: let $K = \begin{bmatrix} k_1 & k_2 \end{bmatrix}$. Then

$$\det(sI - A + BK) = s^2 + (3 + k_2)s + (2 + k_1).$$

Desired poles $-4, -5$ give $s^2 + 9s + 20$, hence $K = \begin{bmatrix} 18 & 6 \end{bmatrix}$.
Observer: let $L = \begin{bmatrix} l_1 \\ l_2 \end{bmatrix}$. Then

$$\det(sI - A + LC) = s^2 + (l_1 + 3)s + (3l_1 + l_2 + 2).$$

Desired poles $-8, -9$ give $s^2 + 17s + 72$, hence $L = \begin{bmatrix} 14 \\ 28 \end{bmatrix}$.

2×2 ARE Hand Pattern

For the double integrator

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$
$$Q = \begin{bmatrix} q_1 & 0 \\ 0 & q_2 \end{bmatrix}, \quad R = r,$$

assume symmetric $P = \begin{bmatrix} p_{11} & p_{12} \\ p_{12} & p_{22} \end{bmatrix}$. The ARE gives

$$q_1 - \frac{p_{12}^2}{r} = 0, \qquad p_{11} - \frac{p_{12}p_{22}}{r} = 0,$$

$$q_2 + 2p_{12} - \frac{p_{22}^2}{r} = 0.$$

Choose the stabilising positive-semidefinite solution and then

$$K = R^{-1}B^T P = \frac{1}{r}\begin{bmatrix} p_{12} & p_{22} \end{bmatrix}.$$

This is the common hand-calculation pattern: substitute symmetric P, equate entries, select the physically valid/stabilising root.

Matrix Calculation Checklist

Rank: row-reduce; rank is number of pivots. For square matrices, nonzero determinant implies full rank.
Eigenvalues: solve $\det(\lambda I - A) = 0$. Continuous stability requires $\Re\lambda < 0$; discrete stability requires $|\lambda| < 1$.
2×2 inverse:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Matrix order: AB exists when inner dimensions match; generally $AB \neq BA$.
Controller check: dimensions $K : m \times n$, so $BK : n \times n$.
Observer check: $L : n \times p$, so $LC : n \times n$.
Always verify: rank tests, desired eigenvalues, sign convention and physical reasonableness.

High-Probability One-Sentence Answers

Why feedback? It compares measured output with the reference so disturbances and modelling errors can be corrected.
Why integral action? It accumulates persistent error and raises system type, removing step steady-state error, but can reduce stability margins.
Why derivative filtering? Derivative action amplifies high-frequency measurement noise, so a low-pass filter is required.
Why controllability first? State feedback cannot move an uncontrollable mode.
Why observability first? An observer cannot reconstruct an unobservable mode from the available outputs.
Observer role? It estimates unavailable states so state-feedback control can be implemented from measured inputs and outputs.
Large R in LQR? Control effort is penalised more, so the controller becomes less aggressive and usually slower.
Lead or lag? Use lead for transient response/phase margin; lag for steady-state accuracy/low-frequency gain.

Integrated Design Workflow

1. Derive a model and state assumptions/valid range.
2. Identify poles, stability and step-response limitations.
3. For simple SISO: PID/lead-lag; check Bode/Nyquist margins and actuator nonlinearities.
4. For state-space/MIMO: check controllability and observability.
5. Choose pole placement for exact dynamics or LQR for a performance-effort optimum.
6. Add an observer when states are not measured; avoid unnecessarily fast observer poles.
7. Add prefilter/integral augmentation for tracking.
8. Validate noise, delay, saturation, uncertainty and sampling—a controller that works only for the nominal linear model is not hardware-ready.

Q7: Kalman Filter — Purpose & Noise Logic

Stochastic model:

$$x_{k+1} = Ax_k + Bu_k + Gw_k, \qquad y_k = Cx_k + Du_k + v_k.$$

Process noise w: disturbances/unmodelled dynamics. Measurement noise v: sensor noise/quantisation. Standard assumptions: zero-mean, white, Gaussian and mutually uncorrelated.

$$Q_w = E[ww^T] \succeq 0, \qquad R_v = E[vv^T] \succ 0.$$

Purpose: recursively estimate unmeasured states while minimising the estimation-error covariance

$$P = E[(x - \hat{x})(x - \hat{x})^T].$$

Kalman vs Luenberger: a Luenberger observer places poles deterministically; a Kalman filter chooses the gain optimally from model and noise covariances.
Trust rule: larger $R_v \Rightarrow$ trust sensors less/model more; larger $Q_w \Rightarrow$ model is less trusted and measurements receive more weight.

Kalman Matrix Workflow — Predict then Update

1. Prediction (a priori):

$$\hat{x}_k^- = A\hat{x}_{k-1} + Bu_{k-1},$$

$$P_k^- = AP_{k-1}A^T + GQ_wG^T.$$

2. Innovation and gain:

$$\tilde{y}_k = y_k - C\hat{x}_k^- - Du_k,$$

$$S_k = CP_k^-C^T + R_v, \qquad L_k = P_k^-C^TS_k^{-1}.$$

3. Correction (a posteriori):

$$\hat{x}_k = \hat{x}_k^- + L_k\tilde{y}_k, \qquad P_k = (I - L_kC)P_k^-.$$

Meaning: prediction propagates both state and uncertainty; the update corrects the model using the measurement residual. P_k^- usually grows between measurements because process noise adds uncertainty.

Steady-State Kalman Filter & Duality

Continuous observer:

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}).$$

Filter Riccati equation:

$$AP + PA^T - PC^TR_v^{-1}CP + GQ_wG^T = 0,$$

$$L = PC^TR_v^{-1}.$$

A stabilising steady-state filter requires (A, G) stabilisable and (A, C) detectable.
LQR–Kalman duality:

$$A \leftrightarrow A^T, \quad B \leftrightarrow C^T,$$
$$Q_{LQR} \leftrightarrow GQ_wG^T, \quad R_{LQR} \leftrightarrow R_v,$$

$$K = R^{-1}B^TP \quad \leftrightarrow \quad L = PC^TR_v^{-1}.$$

Exam sentence: The Kalman filter is an optimal noisy-state estimator; it is the estimation dual of LQR.

LQG, Separation Principle & LTR

LQG: LQR state feedback plus Kalman state estimation:

$$u = -K\hat{x}, \qquad \dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}).$$

Separation principle: design K and L independently; nominal closed-loop poles are

$$\operatorname{eig}(A - BK) \ \cup \ \operatorname{eig}(A - LC).$$

This guarantees nominal stability under the usual controllability/detectability assumptions, but it does **not** preserve LQR's guaranteed gain/phase margins. The estimator adds dynamics and phase lag, so LQG can be fragile to model error.
Loop Transfer Recovery (input LTR): (1) design LQR gain K; (2) choose fictitious process-noise channel $G = B$ and $Q_w = qI$; (3) increase q. As $q \to \infty$, the LQG input-loop approaches the LQR loop and robustness margins are recovered.
Trade-off: aggressive recovery increases estimator bandwidth/gain and can amplify sensor noise or actuator activity.

Robust-Control Backup: H_∞ / Small Gain

$$S = (I + L)^{-1}, \qquad T = L(I + L)^{-1}, \qquad S + T = I.$$

S: tracking/disturbance sensitivity at low frequency.
T: noise/unmodelled-dynamics sensitivity at high frequency.

$$\|M\|_\infty = \sup_\omega \bar{\sigma}(M(j\omega)).$$

Small-gain theorem: feedback of stable M and uncertainty Δ is robustly stable if

$$\|M\|_\infty \, \|\Delta\|_\infty < 1.$$

Common weighted tests:

$$\|W_mT\|_\infty < 1 \qquad \text{(robust stability)},$$
$$\|W_eS\|_\infty < 1 \qquad \text{(performance)}.$$

Intuition: robust design limits the worst-case amplification over all frequencies, rather than optimising only the nominal model.

Q8: MPC Receding-Horizon Workflow

Prediction model:

$$x_{i+1} = Ax_i + Bu_i, \qquad y_i = Cx_i.$$

At each sample:
1. measure/estimate current state x_k;
2. predict behaviour over N future steps;
3. minimise tracking error and control effort subject to constraints;
4. apply only the first optimal input u_0^*;
5. shift the horizon and solve again using new measurements.

Typical finite-horizon cost:

$$J = \sum_{i=0}^{N-1}\left(e_i^TQe_i + u_i^TRu_i\right) + e_N^TP_fe_N,$$
$$e_i = x_i - r_i.$$

For smoother actuators, penalise Δu_i instead of/in addition to u_i. Future references $r_{k+1:k+N}$ give **preview**, so MPC can anticipate turns or setpoint changes.

MPC Quadratic Program & Constraints

Stack future inputs:

$$U = \begin{bmatrix} u_k^T & u_{k+1}^T & \cdots & u_{k+N_c-1}^T \end{bmatrix}^T.$$

After substituting predictions into the quadratic cost:

$$\min_U \ \frac{1}{2}U^THU + f^TU$$

subject to

$$A_{ineq}U \leq b_{ineq}.$$

Constraints may represent

$$u_{min} \leq u_i \leq u_{max}, \quad \Delta u_{min} \leq \Delta u_i \leq \Delta u_{max},$$

$$x_{min} \leq x_i \leq x_{max}, \quad y_{min} \leq y_i \leq y_{max}.$$

$H \succeq 0$ gives a convex QP. Hard constraints must never be violated; soft constraints add slack variables and penalties to preserve feasibility.
Failure modes: infeasible QP, excessive computation, model mismatch and delay. Use suitable horizon/sample time, terminal ingredients, warm starts and fallback control.

MPC vs LQR — Likely Comparison Question

	LQR	MPC
Horizon	infinite	finite, repeatedly shifted
Constraints	not explicit	state/input/rate constraints
Control	$u = -Kx$	online optimisation
Computation	offline; cheap online	QP every sample
Model	normally LTI	LTI/LTV; NMPC possible
Stability	Riccati guarantee	needs suitable terminal design
Preview	limited	uses future references

Connection: unconstrained MPC with a sufficiently long/infinite horizon and correct terminal cost approaches the LQR solution.
Why choose MPC? It explicitly handles physical constraints and predicts future behaviour/reference changes.
Main disadvantage? Online optimisation increases computation and can become infeasible or too slow.

Adaptive Control: Gain Scheduling, MRAC, STR

Gain scheduling: measure an operating variable and switch/interpolate among predesigned controllers. Fast and practical for predictable variation, but essentially open-loop adaptation and transitions need validation.
MRAC: choose a reference model with desired response and adapt controller parameters so plant output tracks it:

$$e = y - y_m, \qquad u = \theta^T\phi,$$

$$\dot{\theta} = -\gamma\frac{\partial e}{\partial \theta}e.$$

MIT rule; Lyapunov laws give stronger guarantees. Direct adaptation; no explicit plant identification.
Self-tuning regulator (STR): repeatedly identify plant parameters online, then redesign the controller using certainty equivalence. Flexible and diagnostic, but computationally heavier and requires persistent excitation.
Selection: predictable measured variation $\to$ gain scheduling; known desired model but uncertain plant $\to$ MRAC; explicit changing model required $\to$ STR.

Reinforcement Learning & ILC — Benefits/Risks

RL learns a policy from interaction to maximise discounted reward:

$$\max_\pi E\left[\sum_{t=0}^{\infty}\gamma^t r_t\right].$$

Q-learning update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\Big[r + \gamma\max_{a'}Q(s', a') - Q(s, a)\Big].$$

Benefits: can learn nonlinear policies without an accurate model, adapt to unknown/changing dynamics and optimise long-term behaviour.
Risks: unsafe exploration, weak stability/constraint guarantees, large data demand, reward misspecification, poor interpretability and simulation-to-real mismatch. Use offline training, safety filters, constraints and a conventional fallback controller.
ILC: for the same finite task repeated many times,

$$u_{j+1}(t) = u_j(t) + Le_j(t).$$

It improves trial-to-trial feedforward performance, but is not general online adaptation to arbitrary new tasks.

Planning Essentials: A*, RRT, Hierarchy

Hierarchy: mission goal $\to$ collision-free path $\to$ timed trajectory $\to$ feedback control. Plan slowly; control faster; safety filtering fastest.
A*:

$$f(n) = g(n) + h(n),$$

where g is actual cost-to-come and h estimates cost-to-go. If $h(n) \leq h^*(n)$ (admissible), A* is optimal; Dijkstra is A* with $h = 0$. A consistent heuristic also satisfies

$$h(n) \leq c(n, n') + h(n').$$

RRT: randomly sample, find nearest tree node, steer a fixed step and add the edge if collision-free. Probabilistically complete but not optimal. RRT* adds neighbour selection/rewiring and is asymptotically optimal.
Key distinction: a path is geometric; a trajectory adds timing/velocity/acceleration and must be dynamically feasible.

CBF Safety & Multi-Agent Consensus

Define safe set

$$\mathcal{C} = \{x : h(x) \geq 0\}.$$

For $\dot{x} = f(x) + g(x)u$, enforce

$$L_fh(x) + L_gh(x)u + \alpha h(x) \geq 0.$$

This is linear in u and can be added to a QP safety filter that minimally modifies a nominal PID/LQR/MPC/RL command. If initially safe and feasible, the condition makes $\mathcal{C}$ forward invariant. Potential fields are simpler but may have local minima and no formal safety guarantee.
Consensus protocol:

$$\dot{x}_i = -\sum_j a_{ij}(x_i - x_j), \qquad \dot{x} = -\mathcal{L}x,$$

$$\mathcal{L} = D - A.$$

For a connected undirected communication graph, agents converge to a common value. Formation control applies consensus to shifted states/desired relative offsets. Delays, switching topology and packet loss can degrade convergence/stability.

Q9: System-Synthesis Answer Framework

Use this order for a design/justify question:
1. **Requirements:** tracking, disturbance rejection, speed, safety, constraints, compute budget.
2. **Model:** TF for simple SISO insight; state-space for MIMO/internal dynamics; state assumptions/uncertainty.
3. **Feasibility:** check stability, controllability and observability; identify required sensors/actuators.
4. **Classical baseline:** PID/lead-lag for transparent, low-cost SISO control; verify step response, GM and PM.
5. **Modern layer:** LQR for MIMO regulation; Kalman/LQG for noisy unmeasured states; MPC for constraints/preview; adaptive/RL only when dynamics vary or are poorly known.
6. **Safety/implementation:** saturation, anti-windup, filtering, sampling delay, rate limits, CBF/emergency fallback.
7. **Validation:** nominal + uncertainty/noise/delay tests, worst cases, Monte Carlo, HIL and fail-safe behaviour.

Strong justification form: Use method X because requirement Y matches its main strength; mitigate limitation Z using safeguard W.

Method Selection — One-Line Justifications

PID	simple SISO plant, low computation, easy commissioning; add anti-windup/filtering.
Lead/lag	reshape transient response or steady-state accuracy with classical margins.
Pole placement	direct transient design when states are available and system controllable.
LQR	systematic MIMO state regulation with performance/effort trade-off.
Kalman/LQG	states unmeasured and sensors noisy; then re-check robustness/LTR.
MPC	hard constraints, rate limits and future reference preview are central.
Adaptive	parameters vary predictably or online identification is required.
RL	model is poor and nonlinear decision policy is valuable; requires safety supervision.
CBF	hard safety set must remain invariant while preserving a nominal controller.

Never ignore: actuator saturation/dead-zone, measurement noise, sampling/computation delay, nonlinear operating range, model uncertainty, sensor failure and communication loss.

Week 3 High-Probability Answers ●

Purpose of a Kalman filter? To combine a model and noisy measurements into a minimum-variance estimate of unmeasured states.

Why can LQG be less robust than LQR? The estimator adds dynamics and phase lag, so the full-state LQR margin guarantees no longer apply.

What does LTR do? It tunes the estimator so the LQG loop approaches the robust LQR loop.

What is receding horizon? Optimise several future inputs, apply only the first, then re-solve using new measurements.

Main MPC strength? Explicit handling of state, input and rate constraints with prediction/reference preview.

RL advantage and risk? It can learn without an accurate model, but exploration may violate safety and stability is usually not guaranteed.

Why use a CBF? It converts a safety requirement into a control constraint that keeps the safe set forward invariant.

Planning vs control? Planning selects a feasible route/reference; feedback control tracks it and rejects disturbances.